

FocusFlow — Tech Stack Audit

Version: 1.0

Date: 2 October 2026

Prices verified: 2 October 2026. Vendors change pricing; re-check before each phase.

Scope: the MVP prototype you are about to build, and whether it makes the production version hard later.

The short version

I checked all eight technologies against current pricing. **Six are right and I recommend keeping them. Two need a caveat, and I have not changed them without telling you first. I am not adding any new service.**

Technology	Verdict	One-line reason
TypeScript	Keep	Free forever, and it catches your AI assistant's mistakes before you run anything
Next.js	Keep	Free, and you can send testers a working link with no install
PostgreSQL	Keep	The actual database standard. Not a vendor product — nothing to be locked into
Neon	Keep, do not switch to Supabase	Supabase's free tier deletes nothing but <i>pauses</i> your database after one idle week, and has no backups at all on free
Prisma	Keep, but pin version 7	Free and open source, but version 8 removed some commands the AI will assume exist
Tailwind CSS	Keep	Free, MIT licence, no account, no limits
Vercel	Keep, but read the warning below	Free and excellent. The catch is one clause in their terms that affects you specifically
date-fns + date-fns-tz	Keep, isolate them	Free forever. Confined to one file so they can be swapped later cheaply

Total MVP cost: \$0. No card required on any of them.

The one thing that needs your attention

Vercel's free plan is for personal, non-commercial use only. I have to flag this because your plan is to test with real people.

What that means in practice:

- Showing FocusFlow to friends and testers who are not paying you is fine on the free plan
- The moment you take money for it — a paid subscription, a paid customer, advertising — you must move to the \$20/month Pro plan
- Also arguably commercial: paying a freelancer to build it

This is not a trap and it costs you nothing to find out now. **For the 10-day prototype, the free plan is appropriate.** When you first earn money, upgrade to Pro that day. That is a \$20/month line item, and it is the single cheapest thing you will ever pay for.

Part 1 — Auditing each technology

Each of the eight gets the same seven questions.

1. TypeScript

What it does. JavaScript with a safety net. JavaScript lets you accidentally put text where a number should go, and the app breaks when you press the button. TypeScript stops you at the point of writing the mistake instead. Since you are not writing the code — your AI assistant is — this is the layer that catches its mistakes before you ever see them.

Question	Answer
Free or open source?	Free, open source. Never a bill.
Free-tier limits	None. No accounts, no usage limits.
What could charge me?	Nothing. It is a language, not a service.
Card required?	No.
Free alternative?	JavaScript, but it will not catch the mistakes for you.
Can I migrate away?	Yes. TypeScript compiles to JavaScript. Removing it is deleting type annotations.

Verdict: keep. No cost, no vendor, no account, real benefit.

Future limitation: none worth worrying about.

2. Next.js

What it does. Turns a folder of text files into a working website. You type one command and it starts a website you can open in your browser. It handles the wiring between screens and the database so you do not have to.

Question	Answer
Free or open source?	Free, open source (MIT licence). Hosting costs money — see Vercel below.
Free-tier limits	The software has none. The free <i>hosting</i> has limits.
What could charge me?	Nothing in the library itself. You pay for hosting, which you chose separately.
Card required?	No.
Free alternative?	Any other web framework — but you would be learning something less well documented.
Can I migrate away?	Yes, but this is the most framework-specific decision in the stack.

Verdict: keep. It is the most-used web framework in the world, which means your AI assistant knows it extremely well. Fewer strange errors while you are learning.

Future limitation you should know about. Next.js is built and optimised for Vercel. Moving to a different host is possible and supported, but you would swap a few deployment settings rather than click a button. **The code itself is portable** — the React parts of Next.js are standard React. If you ever move, you move the hosting, not the application.

How to reduce this risk: keep the framework doing ordinary work. Do not lean on Vercel-specific features. This costs you nothing and keeps the door open.

3. PostgreSQL

What it does. The database itself. It stores your tables and refuses to save nonsense — for example, a task pointing at a goal that does not exist. That refusal is doing real work for you in this project.

Question	Answer
Free or open source?	Free, open source. The most widely deployed database in the world.
Free-tier limits	The database has none. The <i>hosting</i> has some.
What could charge me?	Nothing about PostgreSQL itself. Hosting costs are separate.
Card required?	No.
Free alternative?	SQLite (a file instead of a server), MySQL. Both real, both fine.
Can I migrate away?	Yes, and this is the safest choice in the whole stack.

Verdict: keep.

Why this is your lowest-lock-in decision. PostgreSQL is not a product anyone can take away from you. It is a public standard, and the SQL in your queries works on any host in the world — Neon, Supabase, AWS, Google, a computer in a cupboard. If you ever leave Neon, your application code does not change at all. You change one connection string.

The one trap to avoid: do not use anything specific to Neon, such as their branching feature, in a way your logic depends on. Branching is a nice extra, not a foundation.

4. Neon — the database host

What it does. Runs your PostgreSQL database on their computers, on the internet, so your app can reach it. This is what lets you send someone a link and have them use a real database rather than a fake one on your laptop.

Question	Answer
Free or open source?	The software is free and open source. Neon is a commercial company that hosts it for you.
Free-tier limits	1 GB storage, 100 projects, 100 compute-hours per project per month, 5 GB network transfer, 6-hour restore history, 1 manual snapshot. No expiry date.
What could charge me?	Only if you deliberately upgrade to their paid "Launch" plan (\$0.106 per compute-hour, \$0.35 per GB-month). On the free plan you cannot be charged — hitting a limit pauses things rather than billing you.
Card required?	No. Neon does not ask for a card on the free plan.

Free alternative?	Supabase (compared below), or running PostgreSQL on your own computer.
Can I migrate away?	Yes. See above — you change a connection string.

Verdict: keep. Full comparison with Supabase in Part 2.

Two things to understand about the limits:

Idle costs nothing. Neon's server shuts down after 5 minutes of nobody using it, and wakes up in a few hundred milliseconds when someone does. So a prototype nobody is using costs nothing, and there is no such thing as paying for a forgotten test environment.

Going over a limit pauses rather than bills. If you exceed storage or compute on the free plan, writes fail and the database suspends. It does not charge you. This is a much safer arrangement than a plan that quietly bills you for going slightly over.

Future limitation: if you get genuinely large — more than about 1 GB of data, or needing backups longer than 6 hours — you will move to their \$10-ish Launch tier or another host. That is a billing change, not a migration: the connection string stays the same, per Neon's own documentation.

How much data is 1 GB? A single task row is roughly 500 bytes. Even a very large app holds about two million of them. For your prototype with five testers for ten days, you will use well under one megabyte. You are not going to approach this limit.

5. Prisma

What it does. Sits between your code and PostgreSQL. You write your tables in one readable file in plain English-like lines, and Prisma turns that into a real database and gives you simple functions to read and write. You never write raw database commands.

Question	Answer
Free or open source?	Yes — Apache 2.0, free forever, including for commercial use. Prisma has publicly committed that the ORM stays open source and will not close.
Free-tier limits	None for the ORM. Prisma also sells a hosted database and hosting product, but you do not need those and the ORM is not coupled to them.
What could charge me?	Nothing, unless you choose their paid hosted products. You are not planning to.
Card required?	No.
Free alternative?	Drizzle (lighter, newer) or plain SQL. Both more work for you.
Can I migrate away?	Yes, with effort. See the warning.

Verdict: keep, but pin version 7. Here is exactly why.

Prisma version 8 is being released around now and it is a large rewrite. It removed `db push` and `migrate dev` — the two commands an AI assistant will almost certainly try to use — and those moved to a separate package. Version 7 stays supported with bug and security fixes for 18 months after version 8 goes fully live.

The specific danger: your AI assistant has probably seen years of tutorials. If it installs the newest Prisma and then writes commands that no longer exist, you will spend hours on an error that has nothing to do with your logic.

What to do: in `package.json`, pin `"prisma": "^7"` and `"@prisma/client": "^7"`. That one line removes the entire category of problem. I will set it up on Day 1 so you never encounter it.

Future limitation: Prisma is more tied to your database than most choices here. Migrating to Drizzle later would mean rewriting your database calls. Not a disaster — it is mechanical, and it would touch only the `src/data/` folder, which is exactly why that folder is the only place that talks to the database. This is a known, bounded, moderate cost. **Not worth optimising away now.**

6. Tailwind CSS

What it does. Lets you style screens with short labels instead of long stylesheet rules. It matters here because the day view is a dense timeline, and this is the most reliable way to lay something like that out.

Question	Answer
Free or open source?	Yes, MIT licence. Free for personal and commercial use, forever.
Free-tier limits	None. No account, no limits, no tracking.
What could charge me?	Nothing.
Card required?	No.
Free alternative?	Plain CSS, CSS modules, any CSS framework. All fine.
Can I migrate away?	Yes, easily.

Verdict: keep. It produces plain CSS in the end, so anything that reads CSS can read the output.

Future limitation: styles are written with Tailwind class names throughout your components, so a full migration means touching many files. But it is purely visual — nothing breaks, and it can be done file by file rather than all at once. Low risk.

7. Vercel

What it does. Puts your app on the internet at a real web address so testers can open it. It also does the deployment: you push code, it builds and publishes.

Question	Answer
Free or open source?	Hosting is a paid commercial service with a free tier. Not open source.
Free-tier limits	100 GB transfer, 1 million function calls per month, 50,000 analytics events, 200 projects, 100 deployments per day, 1 hour of logs. 1 GB of file storage if you ever use it.
What could charge me?	Only by upgrading to Pro. On the free plan you cannot purchase extra usage — you get paused when you hit a limit.
Card required?	No, for the free plan.
Free alternative?	Netlify, Cloudflare Pages, Fly.io, Render, or running it on your own computer.
Can I migrate away?	Yes. See below.

Verdict: keep, with the commercial-use clause understood.

Your free-tier limits are not remotely close. One million function calls per month is roughly what a small app uses in a year. For five testers, you are talking about hundreds of calls.

The commercial-use clause, restated. Vercel's terms say Hobby is for "non-commercial personal use," and define commercial use as any deployment used for the financial gain of anyone involved in producing the project. Concretely:

Activity	Allowed on free?
Testing with friends and testers who pay nothing	Yes
Building for yourself	Yes
Accepting money, subscriptions, or advertising	No — Pro required
Paying a contractor to build it	Arguably no — Pro required

If in doubt, ask Vercel. But note the trigger is *earning*, not *having users*.

Migrating away later. Next.js runs on any Node host. Moving means changing build settings and the domain. No application code changes. **The cost of this risk is genuinely low.**

One thing to avoid: do not use Vercel Blob for file storage. It is convenient and it locks you in. Part 3 explains the alternative.

8. `date-fns` and `date-fns-tz`

What it does. Handles the confusing arithmetic of dates, times, and time zones. Without it, a meeting at 2pm silently becomes the wrong time twice a year when the clocks change.

Question	Answer
Free or open source?	Yes, MIT licence. Free forever.
Free-tier limits	None.
What could charge me?	Nothing.
Card required?	No.
Free alternative?	Luxon, or the browser's new native Temporal API. All reasonable.
Can I migrate away?	Yes, cheaply — if you keep them in one file.

Verdict: keep, confined to one file.

Why the confinement matters. These libraries are a common source of subtle bugs because time zones are genuinely hard. If every use of them lives in a single file, swapping to a different library later is one file's work instead of a codebase search. That costs one folder today and removes an entire category of future pain.

Future limitation: the built-in `Temporal` API is heading toward being the browser's own answer to this. When it is fully supported everywhere, `date-fns-tz` becomes less necessary. Confining it means that future swap is contained.

Part 2 — Neon vs Supabase, compared

You asked for this comparison rather than a silent switch, so here it is properly. **I recommend staying with Neon.** The reasoning is below, and it comes down to two specific behaviours that matter a lot for a prototype.

	Neon	Supabase
What it is	Managed PostgreSQL, and nothing else	A whole backend: database, login, file storage, and more
Free price	\$0	\$0
Free database size	1 GB per project	500 MB per project
Free project count	100	2
What happens when idle	Shuts down after 5 min, wakes automatically. No penalty.	Paused after 1 week of low activity. You must unpause it.
Backups on free	6-hour restore window + 1 manual snapshot	None at all
Point-in-time recovery	6 hours, free	Not available on free; \$100/month on Pro
Free network transfer	5 GB per project	5 GB shared across everything
Credit card needed	No	No
Paid entry point	Pay per use, no minimum	\$25/month
Built-in login (auth)	No	Yes
Built-in file storage	No	Yes (1 GB free)
Switching away later	Connection string change	Connection string change

Why I recommend Neon

The idle-pause rule is the deciding factor. Supabase pauses a free project after about a week of low activity. That sounds minor. Consider what it means in practice: your tester opens the link on day nine of a gap, sees an error, and tells you the app is broken. Or you demo to someone important and it does not come up. You will spend an afternoon diagnosing something that is not your code.

Neon has no such behaviour in the same sense. Its server sleeps after five minutes and wakes in a few hundred milliseconds, automatically. **The failure mode of a sleeping server is a slightly slower page.** **The failure mode of a paused database is an error page.** For a prototype whose whole purpose is being demoed to real people, that asymmetry decides it.

Neon has backups on the free tier. Supabase does not. This matters more than it sounds. Your whole plan rests on data typed in by testers. If a schema mistake wipes it, Supabase free has no way back. Neon keeps six hours of history plus one manual snapshot, free.

Two free projects is tight. You want somewhere to experiment without risking the live demo. Neon gives you 100.

What Supabase is genuinely better at

It gives you authentication and file storage built in. That is real value, and it is the argument for it. If your main worry were avoiding extra services, Supabase would win.

But it is not free value. Every feature it bundles is a place your product could become dependent on their version of that feature. Authentication is the worst of these, because migrating away from it later is genuinely painful — it is the one decision in this whole stack that is expensive to reverse.

So I am deliberately splitting them. Take the database from Neon, and handle authentication yourself in a way that stores users in your own database (Part 3). That gives you Supabase's main convenience without its lock-in, and it means auth is a library you can remove, not a service you have migrated off.

Lock-in comparison

Both are ordinary PostgreSQL. **Both are equally easy to leave** — you change one connection string and your data comes with you. This is the real reason both are safe.

The difference is what you build *on top*. If you use Supabase Auth, leaving means migrating your user accounts. If you use plain PostgreSQL with your own user table, leaving means changing nothing. **That is why the recommendation below matters more than the Neon-versus-Supabase choice.**

Part 3 — The eight things the stack does not yet cover

You asked me to be explicit about these. **For each, my recommendation is to add it when you need it, not now.** Adding infrastructure you do not use is a cost with no benefit — more accounts, more bills to watch, more things that can break.

Need	Prototype decision	Add later when	Cost
Authentication	Simple passwordless link — see below	More than about 50 users	\$0 → \$0
Database storage	Neon, already decided	Never, it is set up	\$0
File storage	Nothing. No uploads in the prototype.	You add attachments	\$0 → free tier
Hosting	Vercel free, already decided	You take money	\$0 → \$20/mo
Email	Nothing. Not needed for the prototype.	Onboarding or password resets	\$0
Notifications	Nothing. See the honest note.	Focus mode ships	\$0 → varies
Analytics	Count rows in your own database. See below	You need marketing data	\$0
Backups	Neon free snapshot + a weekly dump	Before real user data	\$0
Secrets	One .env file, free	Always	\$0

Two of these deserve more than a table row.

Authentication — the one to get right now

The expensive mistake here is picking a hosted login service in the prototype and having to migrate off it later. Authentication is the least reversible decision in this entire stack, because every screen depends on knowing who is logged in.

My recommendation: build it yourself, storing users in your own database.

For five testers, this does not need to be complex. You enter an email address, the app emails you a link, clicking it signs you in. That is genuinely all a prototype needs, it is about forty lines of code, and it costs nothing.

The reason to do it now rather than later is the shape of the data, not the difficulty. If your user table is yours, with your columns, then adding proper login screens later is new screens. If you used a hosted service, leaving it is a migration of every user's account.

The library to use: Better Auth. It is MIT licensed, free at any scale, TypeScript-first, and stores users in your own database. Worth knowing: Auth.js, the older recommendation, entered security-patch-only maintenance in September 2025 and its maintainers now point new projects at Better Auth. So do not let an AI assistant install Auth.js.

Option	Free?	Where users live	Leaving later
Better Auth	Free forever, MIT	Your database	Nothing to migrate
Clerk	Free to 50,000 users, then \$25/mo + \$0.02/user	Clerk's servers	Migrate every account
Supabase Auth	50,000 users free	Supabase's database	Migrate every account
Auth.js	Free	Your database	Nothing, but no longer developed

Clerk's free tier is genuinely generous and it is the fastest route if you were ever in a hurry. But it stores your users on their servers and the cost climbs per user forever. For a product where users are individuals, not company seats, owning the table is the better long-term shape.

Card required: none of these. Clerk asks for a card only when you upgrade past the free tier.

File storage — add nothing now

The prototype has no images, PDFs, or attachments. Adding storage for files that do not exist is infrastructure you pay for in complexity and get nothing from.

When you need it: if you add attachments or avatars.

What to use then, and why: Cloudflare R2. It has a permanent free tier of 10 GB, and — this is the important part — **it charges nothing for sending files out over the internet**. Almost every other host charges by the gigabyte you serve, and that is where small businesses get surprised bills.

Cloudflare R2 speaks the same storage language as Amazon S3, so switching to anything else later is a settings change, not a rewrite. Its free tier is not a trial — it does not expire.

Cost: \$0 for a very long time. R2 is \$0.015 per GB-month after the free 10 GB, and no egress charge at any size.

Card required: Cloudflare asks for a card when you set up R2, because R2 has no fully card-free path — but **you are not setting it up now**, so this does not apply to the prototype.

Email — add nothing now

Nothing in the 10-day prototype sends an email. Even the passwordless login can use a temporary trick for testers, or you can share one link among them.

When you need it: real onboarding, password resets, the weekly review.

What to use then: Resend, free tier of 3,000 emails per month and 100 per day, with up to three verified domains. No card needed.

The catch to know: you must own a domain and prove you control it by adding a DNS record. Until then you can only send to your own email address. That is a day of waiting, not a cost.

A trap to avoid: do not sign up for anything that has a "free trial" that quietly becomes a paid subscription. Read the trial length. Plausible, for example, gives 30 days free and then charges — see below.

Notifications — the honest answer

The prototype does not need any notification service, and I am not recommending one.

Here is the reason, and it is worth understanding because it is a common trap. A browser app can show a notification while its tab is open. For notifications when the app is closed — the genuinely useful kind — you need something that wakes the server on a schedule and sends a push message. That means a job scheduler and a push service, which is real infrastructure.

For the prototype, neither is needed. Testers open the app and look at their day. That is the entire test.

One thing worth doing on Day 1 anyway, because it costs nothing: write the reminder logic as an ordinary function that returns a list of "this should be said at 09:00" results. Do not call any service. It will be pure logic, easy to test, and wiring it to a real scheduler later is a small step.

When you do need real notifications, options range from about \$0 (a free tier) to \$20/month. The honest note is that this is the requirement where a browser app is weakest, and where a native app does better.

Analytics — count your own rows

Do not add an analytics service. For your Day 10 test, you need to know how many testers signed up, whether they came back, and how many blocks they moved. **You already store all of that in your own database**, because the whole product is a record of behaviour.

One SQL query gives you the answer, and it costs nothing, has no cookie banner, and collects no data about anyone's visitors.

If you later want traffic analytics for a marketing site, Vercel's built-in analytics is free on the free tier (50,000 events per month) and needs no extra setup. Plausible is the privacy-respecting alternative but has **only a 30-day trial, not a free tier** — so if you ever choose it, remember it starts charging. Avoid services whose "free" offer is a trial.

Backups — free, but do them deliberately

On the prototype: Neon's free tier gives you six hours of restore history and one manual snapshot. That covers accidental deletion of recent data.

Add this, it costs nothing: once a week, download a full copy of your database and keep it somewhere off Neon. Neon's free tier gives you one manual snapshot, and Neon's own tools can produce a standard dump file. A dump file is plain PostgreSQL, so it can be loaded into anything, forever.

Before you have real users, upgrade. Free-tier restore history of six hours is not enough when the data is a user's work. Neon's paid tier extends this and adds longer history. Treat this as a launch-day task, not a prototype task.

Environment variables and secrets

This is the "passwords and settings" file. It holds your database connection string and any API keys.

How it works, in plain terms:

- A file named `.env.local` holds your settings on your own computer. It is listed in `.gitignore`, so it never gets uploaded to GitHub.
- A file named `.env.example` shows which settings are needed without their real values. This one is in GitHub, so other people know what to fill in.
- On Vercel, you add the real values through their dashboard. They stay encrypted there and never appear in your code.

What is in yours right now: exactly one line — the Neon database connection string. That is it. No API keys, no secrets, nothing else.

The rule that matters: if a secret is ever committed to GitHub by accident, treat it as compromised and change it. That is true of every service, not just this one.

Part 4 — The recommended MVP architecture

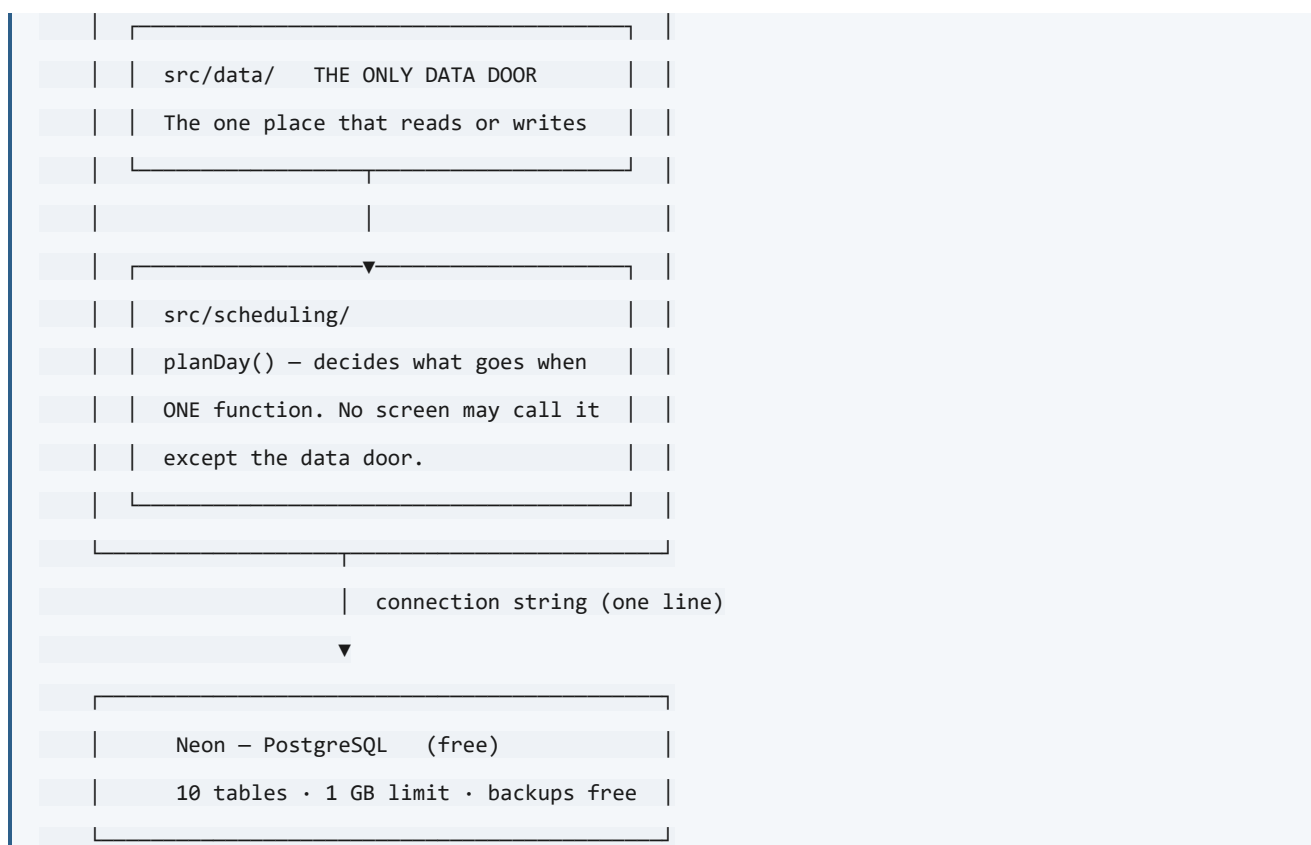
Explained as if you have never seen a technical diagram. Here is the whole system:

What you are actually building

Five parts, and only five:

- 1.
- 2.
- 3.
- 4.
- 5.





What each part does, and what it costs

Part	In one sentence	Cost	Could you remove it?
Screens	The pages people look at. Day view, add a task, add a meeting, first-time setup.	\$0	Never — this is the product
The data door	The only code allowed to talk to the database. Everything else asks it.	\$0	No, and it should never be removed
The scheduler	One function that takes your tasks and time and produces a day. It explains itself.	\$0	Never — this is the thesis
Neon	Where the data lives, on the internet, reachable by your app.	\$0	Yes — change one line of configuration
Vercel	Where the app lives, so other people can open it.	\$0	Yes — move it, the code is unaffected
GitHub	Where your code is saved and versioned, so you can undo mistakes.	\$0	Yes, but do not — it is your safety net

Your total: \$0 per month, no card required anywhere.

Why the data door exists

I want to explain this one clearly, because it is the single most valuable structural decision in the design and it costs nothing.

Screens do not read from the database. They ask the data door, which is a small folder of functions. The scheduler is the only thing that arranges time, and it is one function.

Here is what that buys you. When offline support arrives — the hardest thing on your dropped-features list — you write a new version of the data door that saves on the device first and syncs later. The screens do not change. The scheduler does not change. The database does not change. **You add one folder.**

Without that seam, offline support means editing every screen. With it, offline support means adding a file. That is the difference between a week and a month.

Why the scheduler is one function

Same reason. `planDay()` takes tasks and constraints in, and returns a whole planned day. Nothing else places anything in time.

When you later add calendar sync, routines, and learned energy patterns, they all feed *into* that one function. The scheduler gets more input sources, not more call sites. And when you get it wrong — and you will — there is exactly one place to look.

A second benefit you may not have thought about: it is testable. You can feed the same tasks in twice and check you get the same day. That is what makes the product trustworthy, and it is only possible if scheduling lives in one function.

Why goals and projects tables exist but are empty

Because `Task` already has empty `goalId` and `projectId` columns from day one.

Turning on goals later is new screens pointing at columns that already exist. Had you left the columns out, switching goals on would mean asking every tester to re-file their history — or losing it.

This is the pattern to watch for across the whole design: **pay a tiny cost now so that a future feature is an addition rather than a migration.** Empty columns cost nothing. Reconstructing lost data costs everything.

How this stays portable

Three decisions keep you from being trapped, and each costs nothing:

Decision	What it prevents
Only <code>src/data/</code> talks to the database	Rewriting every screen when storage changes for offline
Only <code>planDay()</code> places anything in time	Rewriting scheduling when features are added
Users stored in your own tables	Migrating every account if you outgrow a login service

And the deeper portability comes from the format choices:

- **PostgreSQL** is a standard, not a product. Any host runs it.
- **Next.js** runs on any Node host. Only deployment settings change.
- **Prisma** targets standard SQL. Every provider supports it.
- **R2, when you need it**, speaks Amazon's storage language, so it is interchangeable.

The one genuine vendor tie is Vercel, because Next.js is optimised for it. The risk is low and the reason is worth restating: you would be moving hosting, not rewriting an application.

Part 5 — What you might eventually pay for

So nothing is a surprise. All figures from 2 October 2026.

During the prototype

Item	Cost
Everything	\$0
Payment card needed	No, anywhere

The realistic path to your first bill

Trigger	What changes	Monthly cost
First tester pays you, or you advertise	Vercel Hobby → Pro. Mandatory under their terms.	\$20
Your database passes 1 GB	Neon Free → Launch, pay per use. Realistically months away.	~\$0–10
Real users with real data	Longer backups on Neon. Worth it.	~\$10
You add file attachments	Cloudflare R2. Free tier is 10 GB.	\$0
You add email	Resend, 3,000 emails free per month.	\$0
You outgrow 50,000 users	Still free — your own auth has no per-user fee	\$0
You need push notifications when the app is closed	Job scheduler plus push service	\$0–20

The honest near-term answer: \$20 per month, triggered by earning your first penny. Everything else is likely to stay at zero well past the prototype.

The one that would hurt

Cloudflare R2 requires a card to set up, unlike everything else here. It has a generous permanent free tier, but the signup asks for payment details. **Do not set it up during the prototype.** There are no files to store yet.

Part 6 — Docker

Agreed: no Docker during the prototype. You are right, and I want to be clear about why rather than just agreeing.

Docker is a way of packaging an app and everything it needs into one box, so it runs identically on any computer. That is genuinely valuable — but it solves a problem you do not have yet. You have one developer, one app, and no production environment that needs to match your laptop exactly. Introducing it now adds a layer to understand and a way for things to break, for no benefit in the next ten days.

But the structure should not block it. Three things make Docker a later addition rather than a rewrite:

- 1.
- 2.
- 3.

How we structure for it without doing it. From Day 1 there will be a `Dockerfile.example` and a note explaining what it would do. Not an active Dockerfile — just the reasoning, written down while it is fresh, so that whoever adds it later does not have to re-derive it.

Hosting portability

Moving off Vercel later means changing build settings and pointing a domain somewhere else. **No application code changes.** Next.js is not tied to Vercel at runtime; the coupling is in deployment convenience, not in the code.

If you later want to leave Vercel and avoid the \$20, the realistic options are Cloudflare Pages and Render. Both are fine. Neither requires code changes.

Part 7 — Decisions for you

Nothing here costs money, and none of it is hard to reverse. I have made a recommendation for each, with the reasoning, so you can overrule any of them.

#	Decision	My recommendation	Why	Cost if wrong
1	Neon vs Supabase	Neon	No idle pausing, 4× the free storage, free backups	A day to switch
2	Auth approach	Build it in, store users in your database	The one expensive thing to reverse later. Better Auth library, free	Hours now, weeks later
3	Prisma version	Pin version 7	Version 8 removed commands your AI will try to use	An afternoon of confusing errors
4	File storage	Add nothing now. R2 when needed.	No files to store yet, and R2 wants a card	Free to fix
5	Analytics service	None. Query your own database.	You already store the data. Plausible's "free" is a trial	Free to fix
6	Email	None now. Resend later, free.	Nothing to send during the prototype	Free to fix
7	Docker	No, but structure for it	No problem to solve yet	Free to fix
8	Auth library	Better Auth, not Auth.js	Auth.js is in security-patch-only maintenance	A migration

Nothing on this list requires a payment method, and I have not enabled billing on anything.

What I need from you

Confirm or overrule the eight. Then two decisions already in the technical design:

- **Consistency score** — I recommend goals advanced, not tasks completed
- **Energy signal** — I recommend a declared curve now, observed learning later, disclosed when added

Once those are settled, **Day 1** begins: the project runs, the database exists, and your settings survive closing the browser.

Appendix — Where the numbers came from

All pricing verified 2 October 2026 against each vendor's own documentation.

Vendor	Source
Neon	Free plan limits and quotas , Plans
Supabase	Pricing , Billing FAQ , Free project pausing
Vercel	Hobby plan , Limits , Fair use guidelines , Analytics limits
Prisma	Pricing , Prisma 8 production readiness , ORM release status , Licence
Cloudflare R2	Pricing
Resend	Pricing , Account quotas
Clerk / Better Auth	Clerk pricing explained , Better Auth vs Clerk vs Auth.js
Plausible	Subscription plans
date-fns	Licence

Two numbers are worth watching. Neon has just raised free storage from 0.5 GB to 1 GB. Clerk recently changed its billing unit and raised its free tier. Both illustrate why this document is dated: re-check before each phase rather than trusting it in six months.